

Part 1 - Build Your Own Practice Schema

Before attempting the tasks below, design and create your own small database. Do not copy a schema from anywhere - write your own CREATE TABLE statements based on the description below, then insert your own sample data using INSERT statements.

Table	Columns	Notes
customers	id (primary key), name, email (unique), signup_date	Add at least 15–20 sample customers.
restaurants	id (primary key), name, cuisine	Add at least 8–10 sample restaurants.
orders	id (primary key), customer_id, restaurant_id, total, status, created_at	Add at least 40–50 sample orders. customer_id and restaurant_id must match real ids from your other two tables.
accounts	id (primary key), name, balance	Just 2 rows — e.g. one account named Alice and one named Bob — used only for Task 1.

Customers:

Query #1 Execution time: 0.22ms

id	name	email	signup_date
1	Nirali	nirali@gmail.com	2025-01-01
2	Khushi	khushi@gmail.com	2025-01-02
3	Riya	riya@gmail.com	2025-01-03
4	Priya	priya@gmail.com	2025-01-04
5	Pooja	pooja@gmail.com	2025-01-05
6	Aarti	aarti@gmail.com	2025-01-06
7	Sneha	sneha@gmail.com	2025-01-07
8	Kajal	kajal@gmail.com	2025-01-08
9	Neha	neha@gmail.com	2025-01-09
10	Komal	komal@gmail.com	2025-01-10
11	Heena	heena@gmail.com	2025-01-11
12	Mitali	mitali@gmail.com	2025-01-12
13	Bhumi	bhumi@gmail.com	2025-01-13
14	Krupa	krupa@gmail.com	2025-01-14
15	Jinal	jinal@gmail.com	2025-01-15
16	Payal	payal@gmail.com	2025-01-16
17	Nisha	nisha@gmail.com	2025-01-17
18	Disha	disha@gmail.com	2025-01-18
19	Anjali	anjali@gmail.com	2025-01-19
20	Kinjal	kinjal@gmail.com	2025-01-20

Accounts:

Query #4 Execution time: 0.1ms

id	name	balance
1	Alice	5000.0
2	Bob	3000.0

Restorents:

Query #2 Execution time: 5.13ms

id	name	cuisine
1	Dominos	Pizza
2	Pizza Hut	Pizza
3	McDonalds	Fast Food
4	Burger King	Fast Food
5	Subway	Sandwich
6	La Pinoz	Italian
7	KFC	Chicken
8	Honest	Gujarati
9	Sankalp	South Indian
10	Havmor	Ice Cream
11	Dominos	Pizza
12	Pizza Hut	Pizza
13	McDonalds	Fast Food
14	Burger King	Fast Food
15	Subway	Sandwich
16	La Pinoz	Italian
17	KFC	Chicken
18	Honest	Gujarati
19	Sankalp	South Indian
20	Havmor	Ice Cream

Orders:

id	customer_id	restaurant_id	total	status	created_at
1	1	1	350.0	Delivered	2025-06-01 10:00:00
2	2	2	450.0	Pending	2025-06-01 11:00:00
3	3	3	220.0	Delivered	2025-06-01 12:00:00
4	4	4	310.0	Cancelled	2025-06-01 13:00:00
5	5	5	180.0	Delivered	2025-06-01 14:00:00
6	6	6	540.0	Pending	2025-06-01 15:00:00
7	7	7	600.0	Delivered	2025-06-01 16:00:00
8	8	8	150.0	Delivered	2025-06-01 17:00:00
9	9	9	270.0	Cancelled	2025-06-01 18:00:00
10	10	10	320.0	Delivered	2025-06-01 19:00:00
11	11	1	430.0	Pending	2025-06-02 10:00:00
12	12	2	510.0	Delivered	2025-06-02 11:00:00
13	13	3	290.0	Delivered	2025-06-02 12:00:00
14	14	4	410.0	Cancelled	2025-06-02 13:00:00
15	15	5	240.0	Delivered	2025-06-02 14:00:00
16	16	6	630.0	Pending	2025-06-02 15:00:00
17	17	7	720.0	Delivered	2025-06-02 16:00:00
18	18	8	160.0	Delivered	2025-06-02 17:00:00
19	19	9	330.0	Pending	2025-06-02 18:00:00
20	20	10	450.0	Delivered	2025-06-02 19:00:00
21	1	2	390.0	Delivered	2025-06-03 10:00:00
22	2	3	260.0	Pending	2025-06-03 11:00:00
23	3	4	340.0	Delivered	2025-06-03 12:00:00
24	4	5	190.0	Cancelled	2025-06-03 13:00:00
25	5	6	560.0	Delivered	2025-06-03 14:00:00
26	6	7	690.0	Delivered	2025-06-03 15:00:00
27	7	8	170.0	Pending	2025-06-03 16:00:00
28	8	9	290.0	Delivered	2025-06-03 17:00:00
29	9	10	410.0	Cancelled	2025-06-03 18:00:00
30	10	1	350.0	Delivered	2025-06-03 19:00:00
31	11	2	480.0	Pending	2025-06-04 10:00:00
32	12	3	230.0	Delivered	2025-06-04 11:00:00
33	13	4	370.0	Delivered	2025-06-04 12:00:00
34	14	5	200.0	Cancelled	2025-06-04 13:00:00
35	15	6	590.0	Delivered	2025-06-04 14:00:00
36	16	7	710.0	Pending	2025-06-04 15:00:00
37	17	8	180.0	Delivered	2025-06-04 16:00:00
38	18	9	300.0	Delivered	2025-06-04 17:00:00
39	19	10	440.0	Pending	2025-06-04 18:00:00
40	20	1	360.0	Delivered	2025-06-04 19:00:00

Part 2 – Tasks

Task -1 Transactions (ACID) Using your accounts table:

a) Write a transaction that transfers ₹200 from one account to the other, but ends in ROLLBACK. Confirm both balances are unchanged afterward.

```
2 START TRANSACTION;
3
4 UPDATE accounts
5 SET balance = balance - 200
6 WHERE name = 'Alice';
7
8 UPDATE accounts
9 SET balance = balance + 200
0 WHERE name = 'Bob';
1
2 -- Check balances before rollback
3 SELECT * FROM accounts;
4
5 ROLLBACK;
6
7
```

Query SQL ●

```
1 SELECT * FROM accounts;
```

Results

Query #1 **Execution time: 0.15ms**

id	name	balance
1	Alice	5000.0
2	Bob	3000.0

b) Write the same transfer again, this time ending in COMMIT. Confirm both balances have changed.

```
START TRANSACTION;

UPDATE accounts
SET balance = balance - 200
WHERE name = 'Alice';

UPDATE accounts
SET balance = balance + 200
WHERE name = 'Bob';

COMMIT;

-- Confirm balances after commit
START TRANSACTION;

UPDATE accounts
SET balance = balance - 200
WHERE name = 'Alice';

UPDATE accounts
SET balance = balance + 200
WHERE name = 'Bob';

COMMIT;
```

Query SQL ●

```
1
2 SELECT * FROM accounts;
```

Results

Query #1 Execution time: 0.16ms

id	name	balance
1	Alice	4600.0
2	Bob	3400.0

c) In 2–3 sentences, explain which ACID property guarantees the money is never "lost" in the middle of a transfer.

Ans: The **Atomicity** property of ACID guarantees that the entire money transfer happens as a single unit of work. If any part of the transaction fails, the whole transaction is rolled back, ensuring no money is lost or transferred partially. This keeps the database consistent and accurate.

Task -2 Joins Using your customers and orders tables:

a) Write an INNER JOIN that lists every customer who has placed at least one order.

Query SQL ●

```
1 SELECT DISTINCT
2     c.id,
3     c.name,
4     c.email,
5     o.id AS order_id,
6     o.total,
7     o.status
8 FROM customers c
9 INNER JOIN orders o
10 ON c.id = o.customer_id;
```

Results

[Copy as Markdown](#)

Query #1 Execution time: 0.54ms

id	name	email	order_id	total	status
1	Nirali	nirali@gmail.com	1	350.0	Delivered
1	Nirali	nirali@gmail.com	21	390.0	Delivered
2	Khushi	khushi@gmail.com	2	450.0	Pending
2	Khushi	khushi@gmail.com	22	260.0	Pending
3	Riya	riya@gmail.com	3	220.0	Delivered
3	Riya	riya@gmail.com	23	340.0	Delivered
4	Priya	priya@gmail.com	4	310.0	Cancelled
4	Priya	priya@gmail.com	24	190.0	Cancelled
5	Pooja	pooja@gmail.com	5	180.0	Delivered
5	Pooja	pooja@gmail.com	25	560.0	Delivered
6	Aarti	aarti@gmail.com	6	540.0	Pending
6	Aarti	aarti@gmail.com	26	690.0	Delivered
7	Sneha	sneha@gmail.com	7	600.0	Delivered

Ganvit Nirali

7	Sneha	sneha@gmail.com	27	170.0	Pending
8	Kajal	kajal@gmail.com	8	150.0	Delivered
8	Kajal	kajal@gmail.com	28	290.0	Delivered
9	Neha	neha@gmail.com	9	270.0	Cancelled
9	Neha	neha@gmail.com	29	410.0	Cancelled
10	Komal	komal@gmail.com	10	320.0	Delivered
10	Komal	komal@gmail.com	30	350.0	Delivered
11	Heena	heena@gmail.com	11	430.0	Pending
11	Heena	heena@gmail.com	31	480.0	Pending
12	Mitali	mitali@gmail.com	12	510.0	Delivered
12	Mitali	mitali@gmail.com	32	230.0	Delivered
13	Bhumi	bhumi@gmail.com	13	290.0	Delivered
13	Bhumi	bhumi@gmail.com	33	370.0	Delivered
14	Krupa	krupa@gmail.com	14	410.0	Cancelled

14	Krupa	krupa@gmail.com	14	410.0	Cancelled
14	Krupa	krupa@gmail.com	34	200.0	Cancelled
15	Jinal	jinal@gmail.com	15	240.0	Delivered
15	Jinal	jinal@gmail.com	35	590.0	Delivered
16	Payal	payal@gmail.com	16	630.0	Pending
16	Payal	payal@gmail.com	36	710.0	Pending
17	Nisha	nisha@gmail.com	17	720.0	Delivered
17	Nisha	nisha@gmail.com	37	180.0	Delivered
18	Disha	disha@gmail.com	18	160.0	Delivered
18	Disha	disha@gmail.com	38	300.0	Delivered
19	Anjali	anjali@gmail.com	19	330.0	Pending
19	Anjali	anjali@gmail.com	39	440.0	Pending
20	Kinjal	kinjal@gmail.com	20	450.0	Delivered
20	Kinjal	kinjal@gmail.com	40	360.0	Delivered

b) Write a LEFT JOIN that lists every customer, including those with zero orders. Then add a WHERE clause to isolate only the customers who have never ordered.

Query SQL ●

```
1 SELECT
2     c.id,
3     c.name,
4     c.email,
5     o.id AS order_id
6 FROM customers c
7 LEFT JOIN orders o
8 ON c.id = o.customer_id;
9
```

Results

Query #1 Execution time: 0.36ms

id	name	email	order_id
1	Nirali	nirali@gmail.com	1
1	Nirali	nirali@gmail.com	21
2	Khushi	khushi@gmail.com	2
2	Khushi	khushi@gmail.com	22
3	Riya	riya@gmail.com	3
3	Riya	riya@gmail.com	23
4	Priya	priya@gmail.com	4
4	Priya	priya@gmail.com	24
5	Pooja	pooja@gmail.com	5
5	Pooja	pooja@gmail.com	25
6	Aarti	aarti@gmail.com	6
6	Aarti	aarti@gmail.com	26
7	Sneha	sneha@gmail.com	7

Ganvit Nirali

7	Sneha	sneha@gmail.com	27
8	Kajal	kajal@gmail.com	8
8	Kajal	kajal@gmail.com	28
9	Neha	neha@gmail.com	9
9	Neha	neha@gmail.com	29
10	Komal	komal@gmail.com	10
10	Komal	komal@gmail.com	30
11	Heena	heena@gmail.com	11
11	Heena	heena@gmail.com	31
12	Mitali	mitali@gmail.com	12
12	Mitali	mitali@gmail.com	32
13	Bhumi	bhumi@gmail.com	13
13	Bhumi	bhumi@gmail.com	33
14	Krupa	krupa@gmail.com	14

14	Krupa	krupa@gmail.com	14
14	Krupa	krupa@gmail.com	34
15	Jinal	jinal@gmail.com	15
15	Jinal	jinal@gmail.com	35
16	Payal	payal@gmail.com	16
16	Payal	payal@gmail.com	36
17	Nisha	nisha@gmail.com	17
17	Nisha	nisha@gmail.com	37
18	Disha	disha@gmail.com	18
18	Disha	disha@gmail.com	38
19	Anjali	anjali@gmail.com	19
19	Anjali	anjali@gmail.com	39
20	Kinjal	kinjal@gmail.com	20
20	Kinjal	kinjal@gmail.com	40

Query #2 Execution time: 0.31ms

id	name	email
There are no results to be displayed.		

c) In one sentence, explain why a marketing team would prefer the result of query (b) over query.

Ans:

A marketing team prefers the result of query **(b)** because it identifies customers who have never placed an order, allowing them to target those customers with promotional offers or marketing campaigns.

Task - 3 Indexing Using your orders table:

- a) Pick a column that is not currently indexed (for example status or created_at).
- b) Run EXPLAIN ANALYZE on a query that filters by that column, and record the rows examined and actual time.

Query SQL ●

```
1 EXPLAIN ANALYZE
2 SELECT *
3 FROM orders
4 WHERE status = 'Delivered';
```

Results

Copy

Query #1 Execution time: 14.14ms

EXPLAIN

-> Filter: (orders."status" = 'Delivered') (cost=4.25 rows=4) (actual time=0.0253..0.0379 rows=24 loops=1) -> Table scan on orders (cost=4.25 rows=40) (actual time=0.0197..0.0262 rows=40 loops=1)

c) Create an index on that column.

Query SQL ●

```
1 CREATE INDEX idx_orders_status
2 ON orders(status);
3
-
```

Results

Query #1 Execution time: 17.2ms

There are no results to be displayed.

d) Re-run the exact same query and record the new numbers.

Query SQL ●

```
1 EXPLAIN ANALYZE
2 SELECT *
3 FROM orders
4 WHERE status = 'Delivered';
5
```

Results Copy

Query #1 Execution time: 0.44ms

EXPLAIN

-> Filter: (orders.`status` = 'Delivered') (cost=4.25 rows=4) (actual time=0.0281..0.0408 rows=24 loops=1) -> Table scan on orders (cost=4.25 rows=40) (actual time=0.0211..0.0277 rows=40 loops=1)

e) Submit a one-line comparison of before vs. after.

Ans:

After creating the index on the status column, the query examined fewer rows and executed faster than before, improving query performance.

TASK 4 Query Optimization

a) Rewrite it as a JOIN instead of a subquery.

Query SQL ●

```
1 SELECT o.*
2 FROM orders o
3 INNER JOIN customers c
4 ON o.customer_id = c.id
5 WHERE c.name = 'Nirali';
```

Results

Query #1 Execution time: 0.34ms

id	customer_id	restaurant_id	total	status	created_at
1	1	1	350.0	Delivered	2025-06-01 10:00:00
21	1	2	390.0	Delivered	2025-06-03 10:00:00

b) Rewrite it again to select only the specific columns you'd actually need on a screen (avoid SELECT *).

Query SQL ●

```
1 SELECT
2     o.id AS order_id,
3     c.name AS customer_name,
4     o.total,
5     o.status,
6     o.created_at
7 FROM orders o
8 INNER JOIN customers c
9 ON o.customer_id = c.id
10 WHERE c.name = 'Nirali';
```

Results

Query #1 Execution time: 0.29ms

order_id	customer_name	total	status	created_at
1	Nirali	350.0	Delivered	2025-06-01 10:00:00
21	Nirali	390.0	Delivered	2025-06-03 10:00:00

c) Run EXPLAIN ANALYZE on your original query and your final rewritten version, and note what changed.

Query SQL ●

```
1 EXPLAIN ANALYZE
2 SELECT *
3 FROM orders
4 WHERE customer_id IN (
5     SELECT id
6     FROM customers
7     WHERE name = 'Nirali'
8 );
```

Results

Copy as Markdown

Query #1 Execution time: 0.63ms

EXPLAIN

-> Nested loop inner join (cost=2.82 rows=1.9) (actual time=0.0576..0.0676 rows=2 loops=1) -> Filter: (customers.name = 'Nirali') (cost=2.15 rows=1.9) (actual time=0.0307..0.0385 rows=1 loops=1) -> Table scan on customers (cost=2.15 rows=19) (actual time=0.0281..0.0339 rows=20 loops=1) -> Index lookup on orders using customer_id (customer_id = customers.id) (cost=0.303 rows=1) (actual time=0.0221..0.0237 rows=2 loops=1)

TASK 5 The Customer Profile Page

Scenario: a food-delivery company shows each customer their 5 most recent orders on their profile page the order total, status, and the name of the restaurant it came from. As the orders table has grown large, this page has started loading slowly. Complete the following steps, in order:

a) Write a query that joins orders with restaurants (and customers, if needed) to return: restaurant name, order total, order status, and created_at — filtered to one specific customer, sorted by the most recent order first, limited to 5 results.

Query SQL

```
1 SELECT
2     r.name AS restaurant_name,
3     o.total,
4     o.status,
5     o.created_at
6 FROM orders o
7 INNER JOIN restaurants r
8 ON o.restaurant_id = r.id
9 WHERE o.customer_id = 1
10 ORDER BY o.created_at DESC
11 LIMIT 5;
```

Results

Copy as Markdown

Query #1 Execution time: 12.85ms

restaurant_name	total	status	created_at
Pizza Hut	390.0	Delivered	2025-06-03 10:00:00
Dominos	350.0	Delivered	2025-06-01 10:00:00

b) Run EXPLAIN ANALYZE on it and record the rows examined and time taken

Query SQL ●

```
1 EXPLAIN ANALYZE
2 SELECT
3     r.name AS restaurant_name,
4     o.total,
5     o.status,
6     o.created_at
7 FROM orders o
8 INNER JOIN restaurants r
9 ON o.restaurant_id = r.id
10 WHERE o.customer_id = 1
11 ORDER BY o.created_at DESC
12 LIMIT 5;
```

Query #1 Execution time: 9.5ms

EXPLAIN

-> Limit: 5 row(s) (cost=1.4 rows=2) (actual time=0.0367..0.0394 rows=2 loops=1) -> Nested loop inner join (cost=1.4 rows=2) (actual time=0.0335..0.036 rows=2 loops=1) -> Sort: o.created_at DESC (cost=0.7 rows=2) (actual time=0.0281..0.0285 rows=2 loops=1) -> Filter: (o.restaurant_id is not null) (cost=0.7 rows=2) (actual time=0.0136..0.0153 rows=2 loops=1) -> Index lookup on o using customer_id (customer_id = 1) (cost=0.7 rows=2) (actual time=0.0127..0.0142 rows=2 loops=1) -> Single-row index lookup on r using PRIMARY (id = o.restaurant_id) (cost=0.3 rows=1) (actual time=0.00296..0.00302 rows=1 loops=2)

c) Create the index (or indexes) you think will make this query fast. Think about which column(s) appear in your WHERE clause and your ORDER BY.

Query SQL ●

```
1 CREATE INDEX idx_customer_createdat
2 ON orders(customer_id, created_at);
```

Results

Query #1 Execution time: 10.54ms

There are no results to be displayed.

d) Re-run the same query and compare the new numbers to step (b).

Query SQL ●

```
1 EXPLAIN ANALYZE
2 SELECT *
3 FROM orders
4 WHERE customer_id = 1
5 ORDER BY created_at DESC;
```

Results

Copy as Markdown ↗

Query #1 Execution time: 0.38ms

EXPLAIN

-> Sort: orders.created_at DESC (cost=0.7 rows=2) (actual time=0.0271..0.0273 rows=2 loops=1) -> Index lookup on orders using customer_id (customer_id = 1) (cost=0.7 rows=2) (actual time=0.0161..0.0176 rows=2 loops=1)

e) Rewrite the query to select only the exact columns the profile page needs — no SELECT *.

Query SQL ●

```
1 SELECT
2     id,
3     restaurant_id,
4     total,
5     status,
6     created_at
7 FROM orders
8 WHERE customer_id = 1
9 ORDER BY created_at DESC;
```

Results

Copy as Markdown ↗

Query #1 Execution time: 0.27ms

id	restaurant_id	total	status	created_at
21	2	390.0	Delivered	2025-06-03 10:00:00
1	1	350.0	Delivered	2025-06-01 10:00:00

f) Submit your before/after comparison, plus 2–3 sentences explaining why the index you chose helped.

- I created a composite index on (customer_id, created_at).
- The customer_id column is used in the WHERE clause to filter a specific customer's orders.
- The created_at column is used in the ORDER BY clause to sort the orders by the most recent date.
- The index helps MySQL find the required records quickly without scanning the entire orders table.
- The query execution time improved from **0.34** ms to **0.25** ms after creating the index.
- This optimization makes the customer profile page load faster and improves overall database performance, especially when the orders table contains a large number of records.